

Enhancement Two: Algorithms and Data Structure Narrative

The artifact I chose to represent algorithms and data structure is my android inventory tracking application created in the java programming language. It is the same artifact from the previous milestone, used in enhancement one. The application was designed to track inventory items in an SQLite database where the user can access inventory items to either read items on a grid view, add new items by name and quantity, update item by quantity, or delete an item. I created this application at the end of 2025 for my mobile architect and programming course at Southern New Hampshire University. This artifact is cohesive and will be modified throughout the course to meet all five course outcomes with the three enhancement categories.

I chose to enhance this artifact with the user in mind. I wanted to improve the user interface (UI) design and improve the user experience (UX). As the inventory increases in size I wanted a way for the user to filter the items alphabetically by item name for better readability as well as by item quantity; either search quantity item from low to high or high to low. This will allow quick filtering on inventory items stock quantity; either low or high in stock which may be useful to inventory tracking. I used collection sort to compare the item to swap based on the specified sorting choice if unsorted the O notation is $O(n \log n)$ and when it is already sorted it is $O(n)$. I programmed the application to refresh the data even after the user adds a new item or updates quantity by creating a `sortUpdate()` method it allowed the items to remain sorted based on the preferred choice keeping the items ordered and allowing better time complexity by not having to re-sort the list each time, since the list will already be sorted by the users preference it will maintain data integrity. This improved code readability by organizing the data. I applied this method to each sorting feature (A-Z, lowHigh, and highLow with a switch statement to handle each sort method as a case). I added a `toLowerCase()` method to the items added in the `insertItem()`

method to remove case sensitivity issues when an item was added to prevent bugs and ensure the items were sorted appropriately eliminating any issues straight from the source. A spinner was utilized for the search items in a form of a drop down keeping the user interface clean and less clutter. An entirely new class InventorySort to handle the sorting feature in its entirety (spinner logic, and each sort method). This enhancement was one of the most time-consuming and file cleanup heavy enhancements of the course. Working out of multiple files to keep the program organized (with separation of tasks) and keeping the program modular.

The primary files for this enhancement were the InventorySort.java and the ActivityInventory.java. With minimal enhancement in the RecyclerView_adapter.java, Activity_inventory.xml, ActivityAddItem.java, and strings.xml. I refactored each file based on the code review. (I also applied formatting corrections to the RecyclerView_list.java although it is not directly linked to the enhancement). This project is designed as a whole and each file upholds the integrity of the program and these files were a part of ensuring the sorting methods would be applied whether it was creating the mechanism for sorting in the xml or being able to access the item name or quantity from the RecyclerView_list, each file had its specified task. This enhancement demonstrates my ability to apply UI/UX design principles, effectively solve problems by writing appropriate methods for reusability and readability, and demonstrate the ability to debug properly while applying techniques that prevented future bugs. Technical proficiency with structuring datasets and algorithms by creating well-defined sorting methods to organize and filter data.

I have met the course outcomes as planned from Module one. This enhancement is ready for my ePortfolio. It covers course outcomes in categories 3 and 4. My files were heavily

reviewed and updated based on the code review, setting up my task to only focus on enhancement 3 in the upcoming module.

The process of enhancing and modifying this task was extensive. I needed to be intentional with my refactoring and adding of new features and classes. What I have learned through this enhancement is the importance of producing readable and modular code. This artifact in general was not a simple task. I designed it from the start to have a MVC design pattern with an adapter class and RecyclerView. It was difficult to implement the logic and understand the full capabilities of what is required of a recycler view (for dynamic grid view) and the importance of the adapter class (to bridge the data logic and the UI components). By designing my application with best android development practices in mind it made this enhancement easier to implement. Adding the new feature was effective and required minimal changes in terms of the xml layout file (adding the UI component), creating the array in the strings.xml (for the spinner items), and the new class InventorySort to handle logic pertaining to all sorting logic. The separation of files really helped organize my work and each specific file had its tasks that really helped me stay focused, on task, and allowed me to add, update, and apply the enhancement without many unexpected errors or bugs. I knew exactly where to look when applying changes or debugging.